

Git Workflow & Basic Commands

효율적인 협업과 깔끔한 히스토리 관리를 위한 가이드

1. 기초 Git 명령어 (Essentials)

프로젝트 시작부터 커밋, 푸시까지 가장 자주 사용되는 기본 명령어 세트입니다.

명령어	설명
git init	현재 디렉토리를 로컬 Git 저장소로 설정
git clone [url]	원격 저장소의 내용을 내 컴퓨터로 복제
git status	파일의 변경 상태(추적 여부, 수정 등) 확인
git add [file]	변경 사항을 Staging Area에 추가 (. 입력 시 전체 추가)
git commit -m "msg"	Staging Area의 파일을 로컬 저장소에 기록
git push origin [name]	로컬 커밋을 원격 저장소에 업로드
git pull origin [name]	원격 저장소의 최신 내용을 가져와 현재 브랜치에 병합
git switch [name]	해당 브랜치로 이동 (구 checkout 대체)

2. 깔끔한 병합 워크플로우: Rebase & Merge

단순 병합보다 히스토리를 일직선으로 유지할 수 있는 권장 워크플로우입니다.

1 로컬 최신화

병합 전, 원격의 최신 상태를 확인합니다.

```
git fetch origin
```

2 내 작업 브랜치(test) 정렬

내 작업 내역을 dev 브랜치의 최신 커밋 뒤로 옮겨 붙입니다.

```
git switch test  
git rebase dev
```

* 충돌 발생 시 수정 후: `git add . -> git rebase --continue`

3 메인 브랜치(dev)로 합치기

정렬된 test의 내용을 dev로 흡수합니다. (Fast-forward)

```
git switch dev  
git merge test
```

4 결과 반영 및 업로드

완성된 최종 dev 상태를 원격에 올립니다.

```
git push origin dev
```



Tip: Rebase 후 강제 푸시

Rebase는 히스토리를 재작성하므로, 이미 원격에 올린 브랜치를 수정했다면 `git push origin test --force`가 필요할 수 있습니다. (단, 다른 사람과 공유하는 브랜치라면 주의!)

3. 핵심 차이 요약

- **Merge:** 두 브랜치를 합쳤다는 'Merge 커밋' 기록을 남기며 통합 (가지가 합쳐지는 모양).
- **Rebase:** 내 커밋들을 대상 브랜치 끝으로 옮겨서, 마치 처음부터 그 지점에서 작업한 것처럼 '일직선 히스토리' 생성.